The Omics Hub

BIOINFORMATICS MASTER CHEAT SHEET

1. Command Line & Bash Core

File & Directory Operations

`pwd`	Print current working directory
`ls -lah`	List all files with human-readable sizes
`cd /dir/`	Change directory
`mkdir -p`	Create directory and parents if needed
`cp -r`	Copy files or directories recursively
`mv`	Move or rename files
`rm -rf`	Remove files recursively (Use with caution!)

File Inspection & Text Parsing

`head -n 20`	View the first 20 lines of a file
`tail -f`	Output appended data as the file grows
`less -S`	View file, do not wrap long lines
`wc -l`	Count the total number of lines in a file
`grep -i`	Search for a pattern (case-insensitive)
`grep -v`	Invert match (exclude lines matching pattern)
`awk '{print $1}'`	Print only the 1st column of a space-delimited file

Data Compression & Transfer

`tar -czvf`	Compress directory into a .tar.gz archive
`tar -xzvf`	Extract a .tar.gz archive
`gzip file`	Compress a file (creates file.gz)
`zcat file.gz`	View contents of a gzipped file without extracting
`rsync -avz`	Securely copy/sync files between servers
`scp file user@host:`	Securely copy file to a remote server

System & Job Management (Slurm)

`htop`	Interactive process viewer
`df -h`	View disk space usage
`du -sh *`	Check folder sizes in current directory
`squeue -u $USER`	View your active Slurm jobs
`scancel <jobid>`	Cancel a specific Slurm job
`sbatch script.sh`	Submit a job to the cluster

`sed 's/old/new/g'`	Replace "old" with "new" globally in text

> **Bash Scripting Best Practices**
>
> Always start scripts with `#!/bin/bash`. Use `set -euo pipefail` to make your script exit on errors, undefined variables, or pipeline failures. Quote your variables (e.g., `"$FILE"`) to prevent word splitting.

2. Conda / Mamba Environment Management

Conda (and its faster drop-in replacement, Mamba) is essential for maintaining reproducible software environments in bioinformatics.

Environment Creation & Activation

`conda create -n env_name`	Create a new, empty environment
`conda create -n env python=3.9`	Create environment with specific Python version
`conda activate env_name`	Activate the environment
`conda deactivate`	Deactivate current environment
`conda env list`	List all installed environments
`conda remove -n env --all`	Completely delete an environment

Package Management

`conda install pkg`	Install a package in active env
`conda install -c bioconda`	Install package from Bioconda channel
`conda search pkg`	Search for available versions of a package
`conda list`	List installed packages in active env
`conda update pkg`	Update a specific package
`conda clean --all`	Remove unused packages and caches (free up disk)

Environment Export & Reproducibility

```
# Export the current environment to a YAML file for sharing
conda env export > environment.yml

# Create a new environment from a shared YAML file
conda env create -f environment.yml
```

3. R & Seurat Core Functions

Tidyverse Data Manipulation

`filter(df, condition)`	Keep rows that match a condition
`select(df, col1, col2)`	Keep only specific columns
`mutate(df, new=x+y)`	Create or modify columns
`group_by(df, col)`	Group data by a categorical variable
`summarize(df, m=mean(x))`	Calculate summary statistics per group
`left_join(x, y, by="id")`	Merge two dataframes by a common key

ggplot2 Visualization

`ggplot(df, aes(x, y))`	Initialize a ggplot object
`geom_point()`	Add a scatterplot layer
`geom_boxplot()`	Add a boxplot layer
`facet_wrap(~ group)`	Create multi-panel plots by group
`theme_minimal()`	Apply a clean, minimal theme
`ggsave("plot.pdf")`	Save the last generated plot

Standard Seurat Workflow (scRNA-seq)

```r
# 1. Initialize and Filter
seurat_obj <- CreateSeuratObject(counts = raw_data, project = "Sample1", min.cells = 3, min.featur
seurat_obj[["percent.mt"]] <- PercentageFeatureSet(seurat_obj, pattern = "^MT-")
seurat_obj <- subset(seurat_obj, subset = nFeature_RNA > 200 & nFeature_RNA < 2500 & percent.mt <

# 2. Normalize and Identify Highly Variable Features
seurat_obj <- NormalizeData(seurat_obj)
seurat_obj <- FindVariableFeatures(seurat_obj, selection.method = "vst", nfeatures = 2000)

# 3. Scale Data and Run PCA
seurat_obj <- ScaleData(seurat_obj)
seurat_obj <- RunPCA(seurat_obj, features = VariableFeatures(object = seurat_obj))

# 4. Clustering and UMAP
seurat_obj <- FindNeighbors(seurat_obj, dims = 1:15)
seurat_obj <- FindClusters(seurat_obj, resolution = 0.5)
seurat_obj <- RunUMAP(seurat_obj, dims = 1:15)

# 5. Visualization & Marker Identification
DimPlot(seurat_obj, reduction = "umap", label = TRUE)
FeaturePlot(seurat_obj, features = c("CD3D", "CD19", "MS4A1"))
markers <- FindAllMarkers(seurat_obj, only.pos = TRUE, min.pct = 0.25, logfc.threshold = 0.25)
```

4. Genes for scRNA-seq Annotation

Standardized marker genes used for cell-type identification in single-cell transcriptomics, with a specialized focus on T-cell immunology and the tumor microenvironment.

Broad Immune Compartments (PBMC / TME)

Cell Type	Canonical Marker Genes
T Cells (Pan)	`CD3D`, `CD3E`, `CD3G`, `TRAC`, `IL32`
B Cells	`CD19`, `MS4A1 (CD20)`, `CD79A`, `CD79B`
Plasma Cells	`SDC1 (CD138)`, `MZB1`, `JCHAIN`, `IGKC`
NK Cells	`NCAM1 (CD56)`, `KLRD1 (CD94)`, `NKG7`, `GNLY`, `FCGR3A (CD16)`
Monocytes (Classical)	`CD14`, `LYZ`, `VCAN`, `S100A9`, `S100A8`
Monocytes (Non-classical)	`FCGR3A (CD16)`, `MS4A7`, `LST1`
Macrophages	`CD68`, `CD163`, `MRC1 (CD206)`, `C1QA`
Dendritic Cells (cDC)	`HLA-DRA`, `CST3`, `CLEC9A` (cDC1), `CD1C` (cDC2)
pDC (Plasmacytoid)	`LILRA4`, `CLEC4C (CD303)`, `TCF4`

Detailed T-Cell Subsets & Exhaustion

Sub-phenotype	Canonical Marker Genes
CD4+ Helper T Cells	`CD4`, `IL7R`, `CCR7` (Naïve/Tcm), `SELL (CD62L)`
CD8+ Cytotoxic T Cells	`CD8A`, `CD8B`, `GZMB`, `PRF1`, `IFNG`
Regulatory T Cells (Treg)	`FOXP3`, `IL2RA (CD25)`, `CTLA4`, `TIGIT`, `IKZF2 (Helios)`
Th1 Cells	`TBX21 (T-bet)`, `IFNG`, `CXCR3`
Th2 Cells	`GATA3`, `IL4`, `IL5`, `IL13`, `PTGDR2 (CRTH2)`
Th17 Cells	`RORC`, `IL17A`, `CCR6`, `KLRB1 (CD161)`
Exhausted T Cells (Tex)	`PDCD1 (PD-1)`, `HAVCR2 (TIM-3)`, `LAG3`, `CTLA4`, `TOX`

Important Note on Annotations

Gene expression can vary highly based on the tissue state (steady-state vs. inflamed vs. tumor). Always validate computational annotations by examining the expression of a combination of markers via FeaturePlot or DotPlot rather than relying on a single gene.